Priority Queues

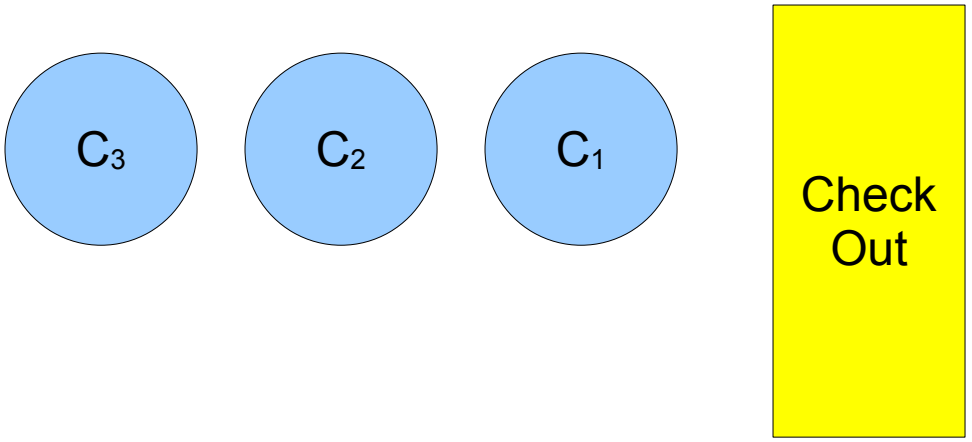
Introduction

- Queuing Theory
- Queues in Operating Systems
- Today’s Assignment

Queuing Theory

- The term queue is from British english and it means an orderly line of people or vehicles
  - In the real world you the most common kind of queue is first come first serve
- In computer science a queue is a data structure containing tasks that can be added to and removed from in some order
  - Most commonly first in first out (FIFO)

Queuing Theory



- FIFO
  - Customer 1 checks out first
  - Then customer 2
  - Finally customer 3

Queuing Theory

- People have tried to figure out how to wait in lines more efficiently for years
- In 1909 Agner Kraup Erlang began formally modeling telephone traffic as a queue called M/D/k.
- In 1930 this algorithm was solved Felix Pollaczek
- In 1932 Aleksandr Khinchin made a statistical solution from Felix's exact one
- This would become the backbone of message and then packet switching for modern computers

Queues in Operating Systems

- Operating Systems are constantly dealing with multiple tasks on finite resources
- As such they have to find ways to order tasks such that each one gets time on the system in some fair amount
- However not all tasks are equally important so operating systems need more advanced queues than simple FIFO ones
- Today we will look at one such queue called a priority queue.

Priority Queue

- A real world example of a priority queue is a hospital emergency room
  - One patient might need attention eventually but other patients might need to be seen to immediately
  - So each patient is assigned some ranking of urgency and tended to in that order
  - If new patients come in with higher urgency patients will lower urgency will be left to wait longer

Priority Queue

- In an operating system some tasks require immediate attention while others can wait
  - Examples
    - Which packets get sent first over a network
    - Which read or write happens first from storage
    - Which jobs should the CPU accomplish first?

Questions?

The Assignment

- Today you will be implementing your own priority queue
- We have provided an outline for what you will need to accomplish
- The priority queue will be a queue of nodes similar to lab 1

Libpriqueue.h

- Libpriqueue.h contains
  - An outline for a node_t struct
  - An outline for the prique_t struct
  - Headers for the node and priority queue functions
- You will not have to edit this file but you may if you want to add other functions
- Note: (*comparer) refers to functions that exist in queuetest.c you do not need to implement these but you do need to handle passing them to the struct

Node helper methods

- node_t *new_node (void *item, node_t *next);
  - mallocs & returns new node with fields filled
- void *destroy_node (node_t *node);
  - frees node & returns item pointer
- void destroy_list (node_t *node);
  - recursively frees node list
- node_t *node_at (node_t *node, int index);
  - gets the node at the given index in the node chain
- int list_size(node_t *node);

Priqueue methods

- void priqueue_init (priqueue_t *q, int(*comparer) (const void *, const void *));
  - This fills the values of a priqueue_t that is passed into it.
  - Set q's comparer to the passed in comparer
  - Set q's top to Null

Priqueue methods

- int priqueue_offer (priqueue_t *q, void *ptr);
  - How we insert data into our queue
  - Find where data *ptr belongs in the queue
  - Create a node with *ptr as the item
  - Connect the node to the queue in the appropriate location.
  - Handle edge cases as they arise

Priqueue methods

- void * priqueue_peek (priqueue_t *q);
  - Returns the pointer item at the head of *q
  - Similar to the peek function on a stack
- void * priqueue_poll (priqueue_t *q);
  - Returns the pointer item at the head of *q
  - Removes the head of *q
  - Similar to the pop function

Priqueue methods

- void * priqueue_at (priqueue_t *q, int index);
  - Returns the item at position in index in priqueue_t *q
  - Returns null if *q does not contain that index
- int priqueue_remove (priqueue_t *q, void *ptr);
  - Removes all instances of *ptr in the priority queue
  - Maintains the remaining queue
  - Returns the number of removed elements

Priqueue methods

- void * priqueue_remove_at(priqueue_t *q, int index);
  - Removes the node at index
  - Maintains the rest of the list
  - Returns the node or null if the node doesn’t exist
- int priqueue_size (priqueue_t *q);
  - Returns the size of *q
- void priqueue_destroy (priqueue_t *q);
  - Destroys and frees all memory associated with *q

queuetest.c

- We have provided a program that will let you test your implementation
- Either invoke it directly or see the makefile for how to call it

Conclusion

- Computers have a vast multitude of tasks they need to complete
- To organize these tasks they use queues
- One such queue is the priority queue which allows the OS to let more important work happen sooner.
- There are several other kinds of queues in modern operating systems and they all try to move work through them as efficiently as possible using some implementation of queue theory